EECS3311 Stage 1 Project Design Report

AI Job Search and Career Agent

Syeda Fatima
219600261

Task 1: Define Your Agent Project

1.1 Project Description

What Problem Does Your Project Solve?
Job seekers must review job postings, compare their qualifications with job requirements, customize resumes, prepare cover letters, practice interviews, and track applications. These activities are often spread across different tools and documents. This makes it difficult to identify suitable opportunities, recognize missing skills, and organize the next steps. The AI Job Search and Career Agent brings these activities into one application. It uses an AI agent to analyze information, retrieve relevant records, compare skills, and produce suggestions for the candidate to review.

Who Are the Target Users?
- University/college students applying for internships or entry-level jobs
- New graduate students seeking their very first job
- Professionals wanting to change their career
- People who want to apply to jobs and be better prepared

What Can the Agent Do?
- Extract different skills and job requirements
- Retrieve content that is previously saved
- Support job matching and comparing
- Draft sample cover letters and resumes
- Create mock interview questions and practice materials
- Develop a career plan for the candidate
- Allows candidate to review proposed ideas before moving forward

Why is an AI Agent Appropriate for This Problem?
The job application and career planning process requires many different steps, such as reading the candidate's profile, looking at their selected jobs, comparing required skills, and identifying career gaps. Therefore, an AI agent is appropriate for this problem because it understands the task and determines what information is needed. The AI Job Search and Career agent requests a tool that is suitable for the problem, executes the task, and decides what the next step is based off of the result. This entire process allows the AI to participate in meaningful agent behaviour such as planning, tool use, retrieval, memory, decision-making, and multi-step task execution.

Which AI/LLM Model(s) Do You Plan to Use?
I plan to use the OpenAI GPT-4.1 mini proposed model, with its identifier configurable through OpenAIAdapter. LLMClient is the provider-independent interface.

How will the AI model interact with the rest of the software system?
The AI model will interact with the rest of the software system through the LLMClient interface. The OpenAIAdapter is what implements the LLMClient interface and deals with the requests and responses given by the candidate or provider. The tools used are:
- ToolManager.readProfile() : retrieve candidate information and skills
- ToolManager.readJob() : retrieve a job and its requirements
- ToolManager.compareSkills() : look at skill matches, gaps, and score
- ToolManager.readNotes() : retrieve company research provided below job description

Overall architecture
JobSearchGUI / JobSearchCLI → AgentController → CareerFacade → domain objects, CareerAgent, ToolManager, and DeadlineManager → Repository, ResumeParser, and LLMClient adapters. Both interfaces share the same application workflows and stored records.

GUI

GUI area	**Responsibility**
Job dashboard and job cards	Import, analyze, compare, and match jobs,
Skills profile	Manage profiles, import resumes, review skills
Application tracker	Manage application records, status, and deadlines
Interview Preparation	Generate practice questions, mock interviews, and feedback
AI Assistant	Resume suggestions, cover letters, research summaries and career plans

CLI

Feature	**Example of Commands**
F01	profile edit
F02	resume import resume.pdf
F03	resume extract-skills
F04	job import
F05	job analyze J01

F06	job gaps J01
F07	job match
F08	job compare J01 J02
F09	resume suggest J01
F10	letter draft J01
F11	interview questions J01
F12	interview start J01
F13	interview feedback S01
F14	application update A01
F15	deadlines list
F16	company notes J01
F17	career plan

1.2 Feature Specification

1. **Feature ID and Name**: F01 – Candidate Profile Management
Description: Creates and updates the information and preferences of the candidate.
User Interaction: User opens the skills profile, edits necessary fields, and clicks save.
Input: Name, highest education level, experience, skills, interests, job location preference
Output: Saved CandidateProfile
AI involvement: Deterministic
Expected Workflow: Enter and validate the field entries → update profile → save → refresh profile
Error/Alternative Cases: Invalid field entries need to be corrected, and a failed save keeps previously stored data.

2. **Feature ID and Name**: F02 – Resume Import
Description: Imports the resume and extracts all of its text.
User Interaction: User imports the resume through the relevant GUI as a PDF or DOCX file.
Input: Supported resume file, which is either a PDF or DOCX.
Output: Stored resume with its extracted text.
AI involvement: Deterministic

Expected Workflow: Select resume file → validate format → extract text → review → confirm file → save file
Error/Alternative Cases: Unsupported, corrupted, or blank files will return an error. Documents that have been scanned and thus don't have extractable text will have the option of copy pasting text as an alternative.

3. **Feature ID and Name**: F03 – Resume Skill Extraction
Description: Identifies skills provided in the resume
User Interaction: User selects Extract Skills option and then reviews proposals and decides whether to edit or accept them.
Input: Previously saved resume text.
Output: Resume evident skills and approved skills update the user profile.
AI involvement: AI-based
Expected Workflow: Get resume → extract proposals → validate resume information → display → candidate reviews → update approved skills to profile
Error/Alternative Cases: Missing resume results in importing again. Skills with no backup support or evidence are dismissed. If AI failure occurs, then skills must be entered by user manually.

4. **Feature ID and Name:** F04 – Job Import
Description: Adds job postings to the workspace application.
User Interaction: User selects Import Job and enters or pastes relevant information
Input: Title, company, job description, salary, location, and optional deadline.
Output: Stored Job
AI involvement: Deterministic
Expected Workflow: Enter details → validate information → check for duplicates → create/update job → save job → display job card
Error/Alternative Cases: Missing information needs to be corrected. Any duplicates will be updated or cancelled.

5. **Feature ID and Name**: F05 – Job Description Analysis
Description: Extracts required skills, experience, and qualifications.
User Interaction: User opens a job card and analyzes it in the relevant GUI area.
Input: Saved job description
Output: All reviewed skills and qualifications with supporting text.
AI involvement: AI-based
Expected Workflow: Get job description → generate analysis → validate information → review → insert accepted analysis → save
Error/Alternative Cases: Missing description prevents analysis generation. Vague or unclear requirements are indicated.

6. **Feature ID and Name**: F06 – Skill Gap Identification
Description: Identifies any skills that are missing, ambiguous, or have gaps
User Interaction: User selects Identify Skill Gaps in the relevant GUI area
Input: Approved skills and job requirements
Output: Missing, uncertain skills, or skill gap explanation report
AI involvement: Hybrid
Expected Workflow: Get extracted skills → compare skills → identify gaps → generate explanation report → display results
Error/Alternative Cases: Missing skills result in manual entry or text extraction. Missing requirements generate analysis report.

7. **Feature ID and Name**: F07 – Job Matching
Description: Rank imported jobs on their compatibility with candidates' profiles and provided skills.
User Interaction: User selects Find Matching Jobs and chooses the top-ranked option
Input: Candidate profile, analyzed jobs, and relevant matching strategy.
Output: Ranked jobs with scores, identified gaps, and explanation
AI involvement: Hybrid
Expected Workflow: Compare skills → identify gaps → calculate policy scores → rank jobs on relevance → generate explanation → display results
Error/Alternative Cases: If no jobs show up, then import again since zero requirements can’t produce a score. Unanalyzed jobs need analysis.

8. **Feature ID and Name**: F08 - Job Comparison
Description: Compare job details and candidate compatibility.
User Interaction: User selects at least 2-4 cards and clicks Compare Jobs
Input: Selected jobs and candidate’s profile
Output: Comparison table and AI-generated summary of results
AI involvement: Hybrid
Expected Workflow: Read jobs → retrieve details → summarize → display results
Error/Alternative Cases: If fewer than 2 jobs are selected, it must be corrected by selecting more. Missing details remain unknown and unaffected.

9. **Feature ID and Name**: F09 – Customized Resume Suggestions
Description: Suggests different ideas for resume improvement for a desired job
User Interaction: User selects Improve My Resume to generate suggestions in the relevant GUI area.
Input: Resume, candidate’s profile, and job analysis
Output: Resume suggestions that are editable and can be stored for career planning
AI involvement: AI-based
Expected Workflow: Get relevant information → generate resume suggestions → check evidence → edit or approve changes → save suggestions

Error/Alternative Cases: Missing resume or job requires proper selection and reimport. Fabricated qualifications are flagged. The original resume is still accessible.

10. **Feature ID and Name**: F10 - Cover Letter Drafting
Description: Drafts a cover letter for candidate's selected job
User Interaction: User selects Generate Cover Letter to draft a letter in the relevant GUI area.
Input: Candidate's profile, resume, job preference, and tone.
Output: Editable cover letter
AI involvement: AI-based
Expected Workflow: Get relevant information → generate cover letter draft → validate letter → display letter → user edits or approves → save document
Error/Alternative Cases: Missing information requires further clarification. Unknown companies or managers receive a formal, generic greeting. Failed generation of the cover letter saves drafts.

11. **Feature ID and Name**: F11 – Interview Question Generation
Description: Generates job-specific, technical, and behavioural questions
User Interaction: User selects Generate Interview Questions in the relevant GUI area
Input: Job requirements and candidates' question count of choice.
Output: Interview questions and evaluation rubric.
AI involvement: AI-based
Expected Workflow: Get information → generate interview questions → validate count and duplicates → save questions → display results
Error/Alternative Cases: Count outside of 1-20 is rejected. Duplicate questions are flagged. If AI failure takes place, then retry.

12. **Feature ID and Name:** F12 – Mock Interview
Description: Generates a mock interview session allowing candidates to better prepare for selected jobs.
User Interaction: User selects Start Mock Interview and answers questions and clicks finish or pause.
Input: Selected job and candidate's answers
Output: *InterviewSession* which contains mock interview questions, answers, and results.
AI involvement: Hybrid
Expected Workflow: Generate questions → create mock interview session → display question → store candidate's answer → request results → save
Error/Alternative Cases: Blank answers to questions require reanswer or skipping. Pausing the mock session saves progress.

13. **Feature ID and Name:** F13 – Interview Answer Feedback
Description: Evaluates answers that are given by the candidate to the interview session.
User Interaction: User selects Get Feedback after submitting answers in the relevant GUI area.
Input: Stored interview question, candidate's answer, evaluation rubric, and job requirements.

Output: Candidate's strengths and weaknesses, improvement suggestions, and overall score according to rubric.
AI involvement: AI-based
Expected Workflow: Get answers → evaluate according to rubric → validate feedback → attach to interview session → display results.
Error/Alternative Cases: Blank or skipped answers are not evaluated. AI failure results in no feedback.

14. **Feature ID and Name:** F14 – Application Tracking
Description: Keeps a record of the candidate's applications and status history.
User Interaction: A record is added or updated in Application Tracker in the relevant GUI area.
Input: Selected job, application submission date, and requested status
Output: Application tracker with job status, updated *Application*
AI involvement: Deterministic
Expected Workflow: Create or retrieve application → validate transition and dates → update status history → save and refresh application tracker
Error/Alternative Cases: Duplicate applications open existing application records. Invalid transitions are denied.

15. **Feature ID and Name:** F15 – Deadline Tracking
Description: Displays upcoming or past deadlines and notifies the other interfaces
User Interaction: Check for deadlines in the relevant GUI area
Input: Application deadlines, current time, and timezone
Output: List of upcoming or overdue applications and reminders
AI involvement: Deterministic
Expected Workflow: Read deadlines → compare to current time → notify candidates →
Error/Alternative Cases: Invalid dates need to be corrected. Missing deadlines are considered unknown. Past or completed applications have no reminders.

16. **Feature ID and Name:** F16 – Company Research Notes
Description: Saves a company's notes and provides a summary of their research
User Interaction: User enters the company's notes and given sources in Company Notes and generates a summary if desired.
Input: Company title, the user's research text, and references
Output: Stored notes in *CompanyNotes* and research summary
AI involvement: Hybrid
Expected Workflow: Add notes and sources → save information → retrieve notes for AI agent to summarize → validate sources → review summary → save
Error/Alternative Cases: Missing company title needs to be corrected. AI failure will result in manual editing from the user.

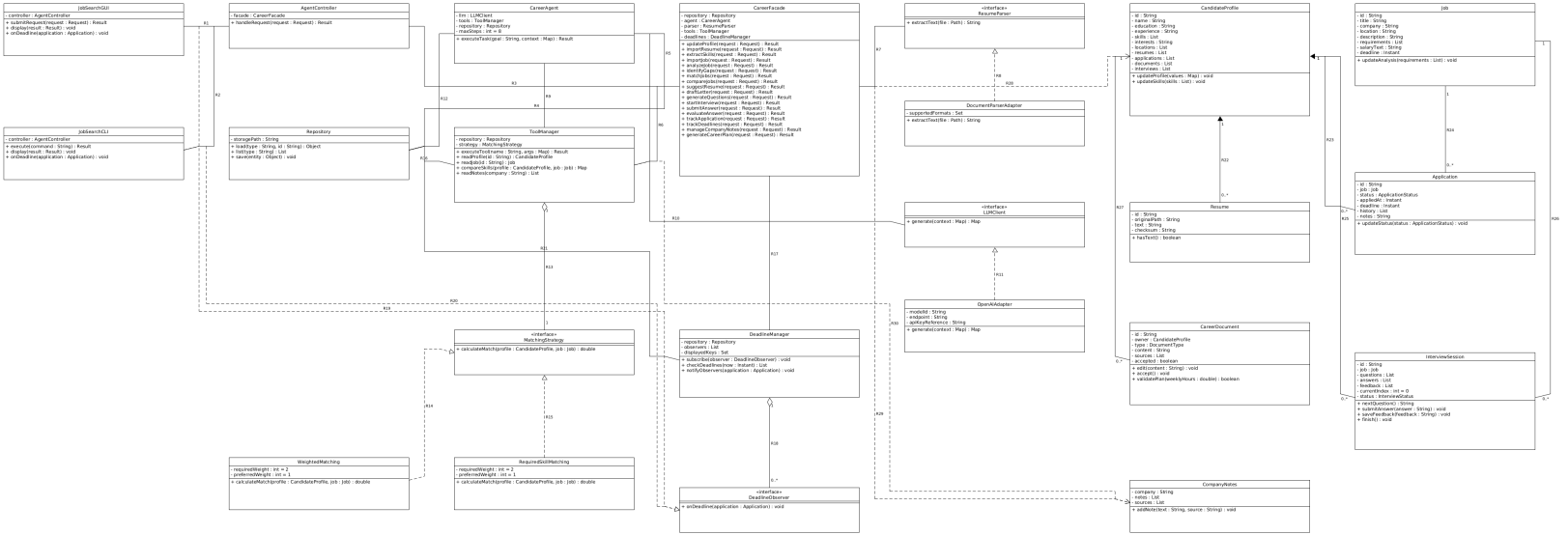
17. **Feature ID and Name:** F17 – Career Development Plan
Description: Suggests different learning activities that address career goals and skill gaps
User Interaction: User selects Generate Career Plan and enters suitable study time.
Input: Candidate's profile, career goals, job gaps, and weekly hours
Output: Reviewed career plan for candidate with learning activities and goals
AI involvement: AI-based
Expected Workflow: Retrieve profile and gaps → generate activities → confirm hours → revise proposals → review and save changes
Error/Alternative Cases: Ambiguous career goals require further explanation. Career plans that are overpriced or overbudget require further assessment.

Task 2: Design Your System Using UML

2.1 Class Diagram

Class/interface	Responsibility
JobSearchGUI	Collect GUI input, display results, receive deadline notifications
JobSearchCLI	Parse commands, display results, receive deadline notifications
AgentController	Route requests to the facade
CareerFacade	Coordinate all seventeen feature workflows
Repository	Load, list, and save records
CareerAgent	Execute the bounded planning/tool-use loop
ToolManager	Validate and execute permitted tools
LLMClient	Define the application’s model-generation contract
OpenAIAdapter	Implement provider-specific model communication
ResumeParser	Define resume text extraction
DocumentParserAdapter	Adapt PDF/DOCX parsing libraries
MatchingStrategy	Define job-scoring behavior
WeightedMatching	Calculate weighted skill coverage
RequiredSkillMatching	Apply required-skill filtering and scoring
CandidateProfile	Store candidate details and verified skills
Resume	Store imported resume text and file information
Job	Store posting information and analyzed requirements
Application	Store status, notes, deadlines, and history
CareerDocument	Store generated content and acceptance state
InterviewSession	Store questions, answers, feedback, and progress
CompanyNotes	Store supplied company research and sources
DeadlineManager	Check deadlines and notify subscribers
DeadlineObserver	Define notification behavior

AI Job Search and Career Agent - Combined Class Diagram



RELATIONSHIP KEY
R1: JobSearchGUI -> AgentController (association)
  submits requests
R2: JobSearchCLI -> AgentController (association)
  submits requests
R3: AgentController -> CareerFacade (association)
  delegates
R4: CareerFacade -> Repository (association)
  persists
R5: CareerFacade -> CareerAgent (association)
  AI work/fixes
R6: CareerFacade -> ToolManager (association)
  uses
R7: CareerFacade -> ResumeParser (association)
  resume parsing
R8: DocumentParserAdapter -> ResumeParser (realization)

RELATIONSHIP KEY
R9: CareerAgent -> ToolManager (association)
  uses tools
R10: CareerAgent -> LLMClient (association)
  inference
R11: OpenAIAdapter -> LLMClient (realization)
R12: CareerAgent -> Repository (association)
  persists
R13: ToolManager -> MatchingStrategy (aggregation)
  selected policy
R14: WeightedMatching -> MatchingStrategy (realization)
R15: RequiredSkillMatching -> MatchingStrategy (realization)
R16: ToolManager -> Repository (association)
  retrieval

RELATIONSHIP KEY
R17: CareerFacade -> DeadlineManager (association)
  deadline tracking
R18: DeadlineManager -> DeadlineObserver (aggregation)
  subscribers
R19: JobSearchGUI -> DeadlineObserver (realization)
R20: JobSearchCLI -> DeadlineObserver (realization)
R21: DeadlineManager -> Repository (association)
  loads deadlines
R22: CandidateProfile -> Resume (composition)
  resume versions
R23: CandidateProfile -> Application (association)
  tracks
R24: Application -> Job (association)
  for job

RELATIONSHIP KEY
R25: CandidateProfile -> InterviewSession (composition)
  owns sessions
R26: InterviewSession -> Job (association)
  prepares for
R27: CareerDocument -> CandidateProfile (association)
  owner
R28: CareerFacade -> CandidateProfile (dependency)
  creates/updates domain objects
R29: CareerFacade -> CompanyNotes (dependency)
  creates and updates notes; manageCompanyNotes()
R30: ToolManager -> CompanyNotes (dependency)
  retrieves stored notes; readNotes()

Notation + public, - private, dashed hollow triangle: realization; hollow diamond: aggregation; filled diamond: composition.
Same 29 classes and 30 relationships as the report. Shared Request, Result and enum value types are referenced in signatures.

Design Patterns

Strategy

What design problem does the pattern address?
Users need different job-matching policies.

Which classes participate in the pattern? What role does each class play?
MatchingStrategy : contract
WeightedMatching, RequiredSkillMatching : implementations
ToolManager : context for selected policy

Why is this pattern appropriate?
This pattern is appropriate because it changes the policy without affecting the workflow.

What would be more difficult if the pattern were not used?
If the pattern was not used, then policy conditions become unorganized.

Observer

What design problem does the pattern address?
Deadline reminders must reach an active interface.

Which classes participate in the pattern? What role does each class play?
JobSearchGUI, JobSearchCLI: concrete observers
DeadlineManager : subject
DeadlineObserver : contract

Why is this pattern appropriate?
This pattern is appropriate because deadline reminder notifications are independent.

What would be more difficult if the pattern were not used?
If the pattern was not used, then the deadline rules would become too dependent on the GUI and CLI.

Adapter

What design problem does the pattern address?
External LLM APIs and parsing use different interfaces.

Which classes participate in the pattern? What role does each class play?
LLMClient, ResumeParser : targets interfaces

OpenAIAdapter, DocumentParserAdapter : adapters

Why is this pattern appropriate?
This pattern is appropriate because adapters can translate external APIs and isolates library formats.

What would be more difficult if the pattern were not used?
If the pattern was not used, then replacing a parser or provider would affect the logic of the application.

Facade

What design problem does the pattern address?
Features require a lot of collaboration amongst the components.

Which classes participate in the pattern? What role does each class play?
CareerFacade : facade
CareerAgent, ToolManager, ResumeParser, DeadlineManager, Repository, and DeadlineManager : subsystems

Why is this pattern appropriate?
This pattern is appropriate because it provides a single clear operation for every feature.

What would be more difficult if the pattern were not used?
If the pattern was not used, then the controller would have to distinguish dependencies and internal systems.

MVC

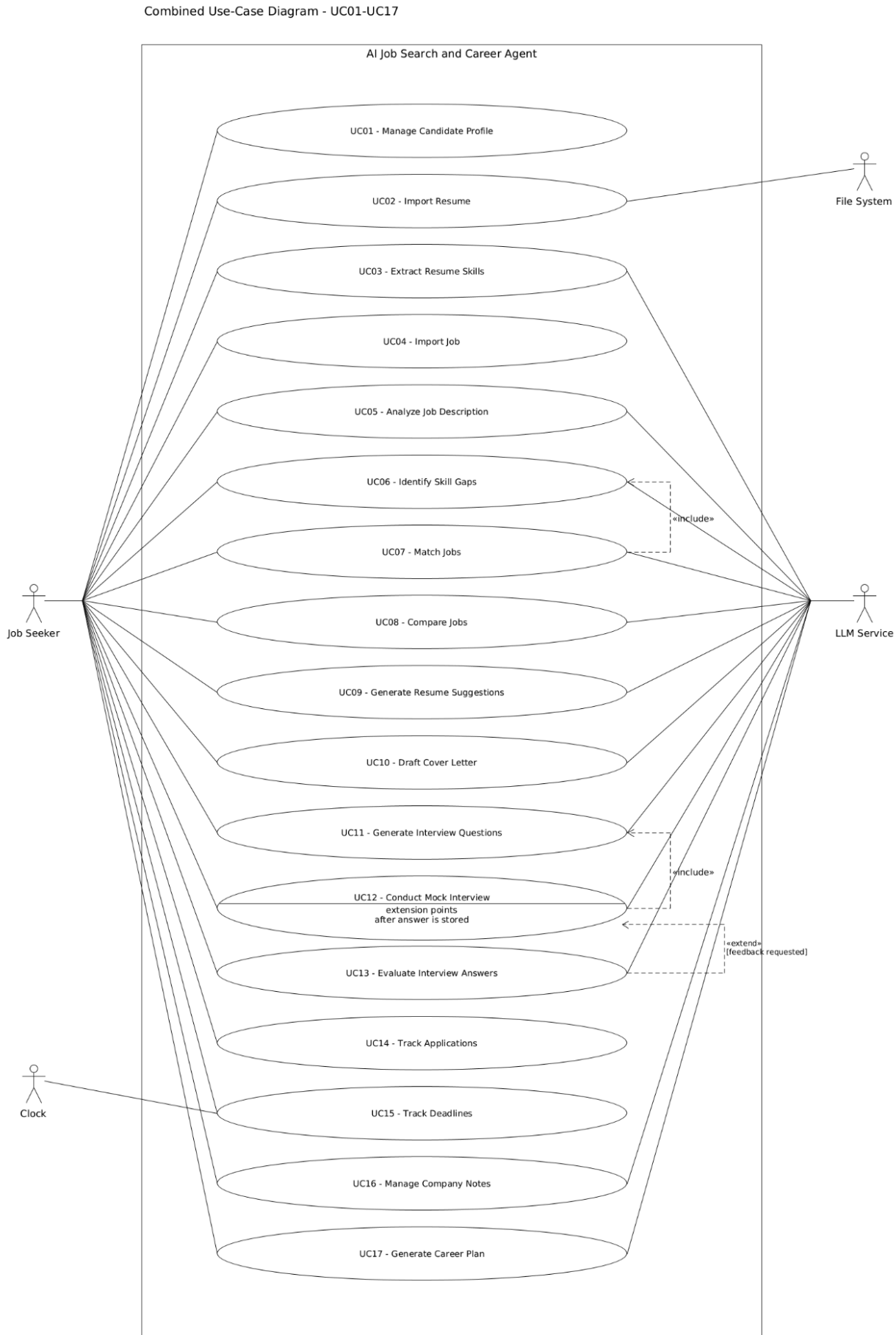
What design problem does the pattern address?
The GUI and CLI need to have consistent behaviour and keep business aspects separate from the interface.

Which classes participate in the pattern? What role does each class play?
JobSearchGUI, JobSearchCLI : views
AgentController : controller
CareerFacade, Repository : provide model and application logic

Why is this pattern appropriate?
This pattern is appropriate because it separates the business rules from the presentation and allows for consistency.

What would be more difficult if the pattern were not used?
If the pattern was not used, then validation and the expected workflow could possibly duplicate.

2.2 Use-Case Diagram



2.2 Use-Case Description

1. **Use-Case ID and Use-Case Name**: UC01 – Manage Candidate Profile
Actor(s): Job seeker
Goal: Maintain candidate's profile
Preconditions: Workspace has to be open
Trigger: Save Profile
Main Success Scenario: Enter details → validate → update → save.
Alternative/Exception Flows: Invalid fields are corrected, and failure to save still preserves data.
Postconditions: Updated profile stored.
Related Features: F01

2. **Use-Case ID and Use-Case Name**: UC02 – Extract Resume Skills
Actor(s): Job seeker, File system
Goal: Store usable resume text
Preconditions: Profile must exist, and the file should be readable.
Trigger: Import Resume
Main Success Scenario: Select file → extract → preview → confirm → save
Alternative/Exception Flows: Invalid fields are corrected, and failure to save still preserves data.
Postconditions: Resume text stored.
Related Features: F02

3. **Use-Case ID and Use-Case Name:** UC03 – Import Resume
Actor(s): Job seeker, LLM service
Goal: Obtain verified skills
Preconditions: Text-bearing resume exists
Trigger: Extract Skills
Main Success Scenario: Retrieve resume → propose skills → validate → review → accept
Alternative/Exception Flows: Missing resume requires reimport, and manual entry is permitted.
Postconditions: Accepted skills are stored
Related Features: F03

4. **Use-Case ID and Use-Case Name**: UC04 – Import Job
Actor(s): Job seeker
Goal: Store job posting
Preconditions: Workspace has to be open
Trigger: Import Job
Main Success Scenario: Enter details → validate → check duplicate → save
Alternative/Exception Flows: Missing job description is rejected, and duplicate offers can be updated or cancelled.
Postconditions: Job is stored

Related Features: F04

5. **Use-Case ID and Use-Case Name**: UC05 – Analyze Job Description
Actor(s): Job seeker, LLM service
Goal: Structure requirements
Preconditions: Job description must exist
Trigger: Analyze Job
Main Success Scenario: Retrieve description → analyze → validate → review → save
Alternative/Exception Flows: Ambiguous entries are flagged, and AI failure permits retry.
Postconditions: Requirements are attached to the job
Related Features: F05

6. **Use-Case ID and Use-Case Name**: UC06 – Identify Skill Gaps
Actor(s): Job seeker, LLM service
Goal: Identify any missing or uncertain skills
Preconditions: Profile skills and an analyzed job must exist
Trigger: Identify Skill Gaps
Main Success Scenario: Retrieve profile and job → compare → classify gaps → generate explanation
Alternative/Exception Flows: Missing analysis is requested, and uncertainty is retained.
Postconditions: Skill gap report is displayed
Related Features: F06

7. **Use-Case ID and Use-Case Name**: UC07 – Match Jobs
Actor(s): Job seeker, LLM service
Goal: Rank compatible jobs
Preconditions: Profile and analyzed jobs must exist
Trigger: Find Matching Jobs
Main Success Scenario: Retrieve jobs → identify gaps → score → rank → generate explanation
Alternative/Exception Flows: No jobs require reimport, and AI failure permits numeric results.
Postconditions: Ranked results are displayed
Related Features: F07

8. **Use-Case ID and Use-Case Name**: UC08 – Compare Jobs
Actor(s): Job seeker, LLM service
Goal: Compare opportunities
Preconditions: 2-4 jobs must be selected
Trigger: Compare Jobs
Main Success Scenario: Validate selection → retrieve → align fields → compare → generate summary
Alternative/Exception Flows: Invalid count is rejected, and unknown fields are retained.
Postconditions: Comparison is displayed

Related Features: F08

9. **Use-Case ID and Use-Case Name**: UC09 – Generate Resume Suggestions
Actor(s): Job seeker, LLM service
Goal: Improve resume relevance
Preconditions: Resume and an analyzed job must exist
Trigger: Improve My Resume
Main Success Scenario: Retrieve evidence → suggest → validate → review → save
Alternative/Exception Flows: Missing resume requires reimport, and unsupported claims are rejected.
Postconditions: Accepted suggestions are stored
Related Features: F09

10. **Use-Case ID and Use-Case Name**: UC10 – Draft Cover Letter
Actor(s): Job seeker, LLM service
Goal: Create an accurate cover letter draft
Preconditions: Profile and analyzed job must exist
Trigger: Generate Cover Letter
Main Success Scenario: Collect context → draft → validate → edit → accept
Alternative/Exception Flows: Missing facts require additional clarification, and AI failure preserves drafts.
Postconditions: Accepted letter stored
Related Features: F10

11. **Use-Case ID and Use-Case Name**: UC11 – Generate Interview Questions
Actor(s): Job seeker, LLM service
Goal: Prepare relevant questions
Preconditions: Analyzed job must exist
Trigger: Generate Questions
Main Success Scenario: Retrieve requirements → generate questions → validate count and duplicates → save
Alternative/Exception Flows: Invalid count is rejected, and AI failure permits a retry.
Postconditions: Question set is available
Related Features: F11

12. **Use-Case ID and Use-Case Name**: UC12 – Conduct Mock Interview
Actor(s): Job seeker, LLM service
Goal: Practice and prepare for an interview
Preconditions: Selected analyzed jobs must exist
Trigger: Start Mock Interview
Main Success Scenario: Generate questions → create session → ask question → store answer → optionally evaluate → continue

Alternative/Exception Flows: User can skip or retry blank answers, and pausing preserves progress.
Postconditions: Session progress saved
Related Features: F12

13. **Use-Case ID and Use-Case Name**: UC13 – Evaluate Interview Answers
Actor(s): Job seeker, LLM service
Goal: Receive feedback
Preconditions: Nonempty answers are stored
Trigger: Get Feedback
Main Success Scenario: Retrieve answers and evaluation rubric → evaluate → validate → attach
Alternative/Exception Flows: Skipped answers are not for score, and AI failure leaves feedback pending.
Postconditions: Available feedback is saved and stored
Related Features: F13

14. **Use-Case ID and Use-Case Name**: UC14 – Track Applications
Actor(s): Job seeker
Goal: Maintain status and history
Preconditions: Selected job must exist
Trigger: Add/Update Application
Main Success Scenario: Create and load application → validate transition → update history → save
Alternative/Exception Flows: Duplicate application opens existing record, and invalid transition is rejected.
Postconditions: Application updated
Related Features: F14

15. **Use-Case ID and Use-Case Name**: UC15 – Track Deadlines
Actor(s): Job seeker, Clock
Goal: Monitor deadlines
Preconditions: Applications must exist
Trigger: Open Deadlines or Clock Check
Main Success Scenario: Retrieve deadlines → compare time → display → notify candidate
Alternative/Exception Flows: Missing dates are unspecified, and repeated notifications are reduced.
Postconditions: Deadline preview is refreshed
Related Features: F15

16. **Use-Case ID and Use-Case Name:** UC16 – Manage Company Notes
Actor(s): Job seeker, LLM service
Goal: Organize company research
Preconditions: Company information must be available
Trigger: Save/Summarize Notes

Main Success Scenario: Enter sources and notes → save → optionally summarize → review
Alternative/Exception Flows: AI failure permits manual notes, and unsupported claims are omitted.
Postconditions: Notes and summary are stored
Related Features: F16

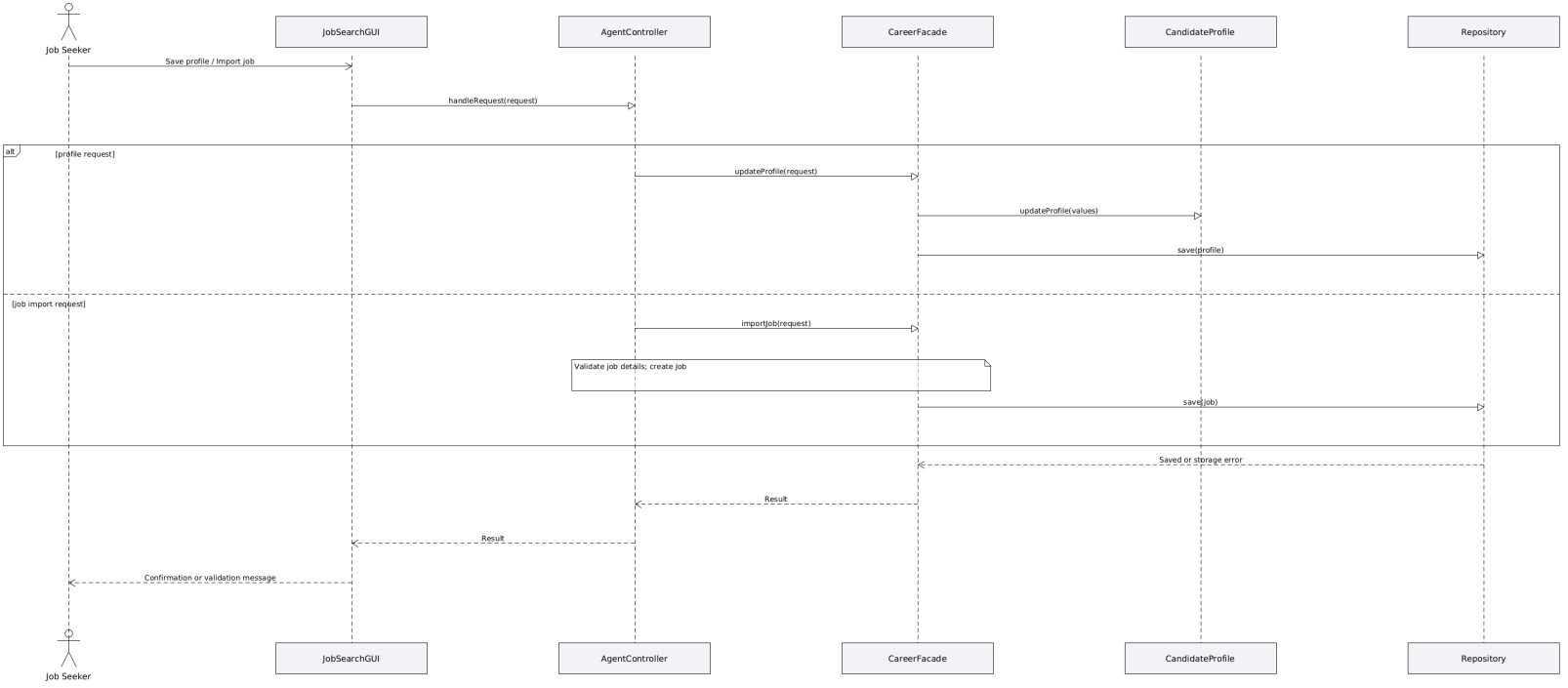
17. **Use-Case ID and Use-Case Name**: UC17 – Generate Career Plan
Actor(s): Job seeker, LLM service
Goal: Create practical career goals
Preconditions: Profile, goals, and budget must exist
Trigger: Generate Career Plan
Main Success Scenario: Collect goal and budget → retrieve gaps → propose tasks → validate → revise → accept
Alternative/Exception Flows: Clarify unclear goals, and over-budget plans cannot be accepted and must be replanned.
Postconditions: A feasible plan is stored
Related Features: F17

2.3 Sequence Diagrams

- SD00 - Profile Management and Job Import
- SD01 - Resume Import and Skill Extraction
- SD02 - Job Analysis, Gaps, Matching, and Comparison
- SD03 - Resume Suggestions and Cover-Letter Drafting
- SD04 - Questions, Mock Interview, and Feedback
- SD05 - Applications and Deadlines
- SD06 - Company Notes and Career Plan
- SD07 - Agent Planning, Tools, and Memory

SD00 - Profile Management and Job Import

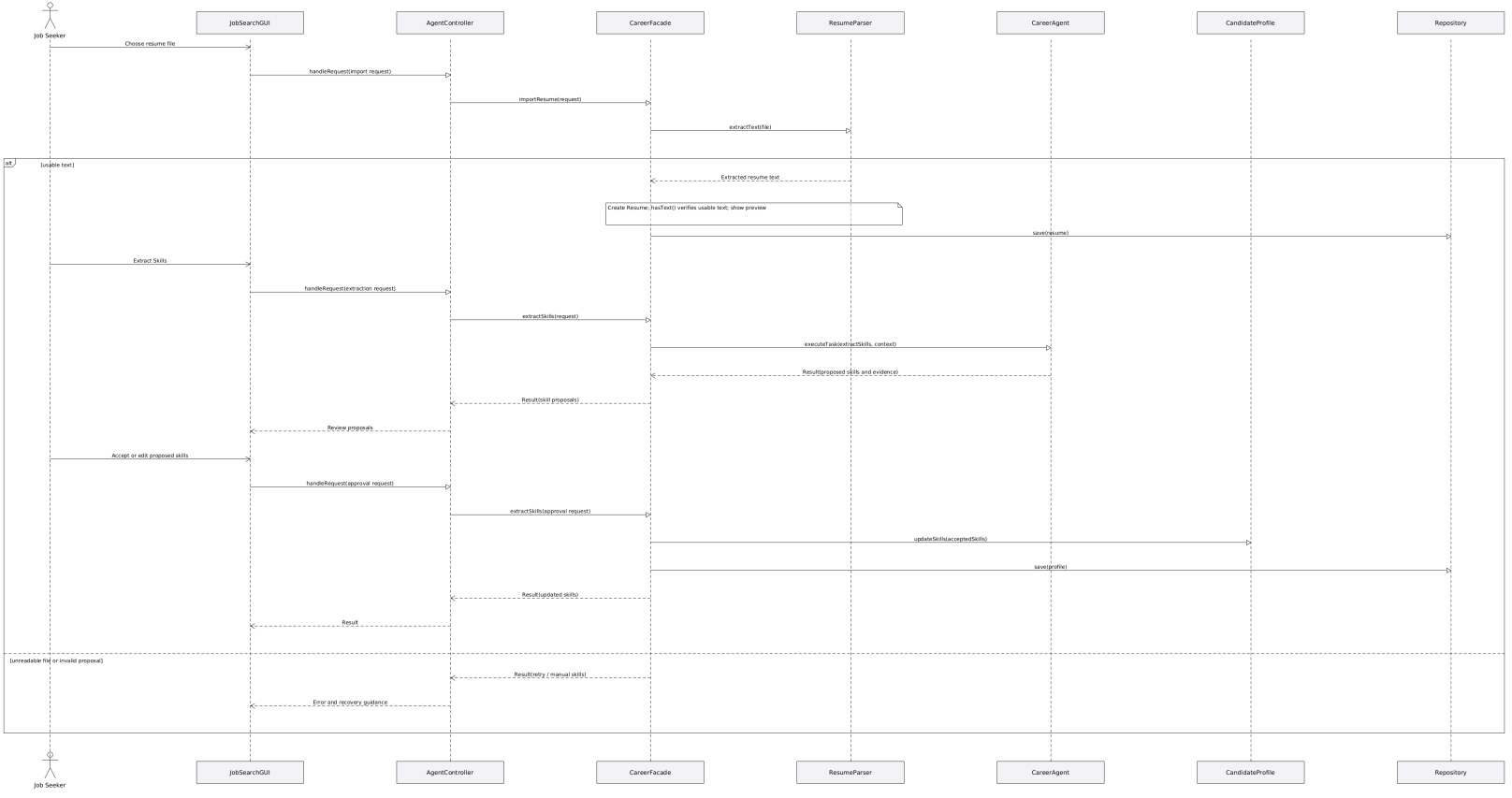
F01, F04. Inputs are validated before saving. Persistence errors preserve previous records.



Solid arrows: calls / user actions. Dashed arrows: returns. CLI uses the same controller and workflows.

SD01 - Resume Import and Skill Extraction

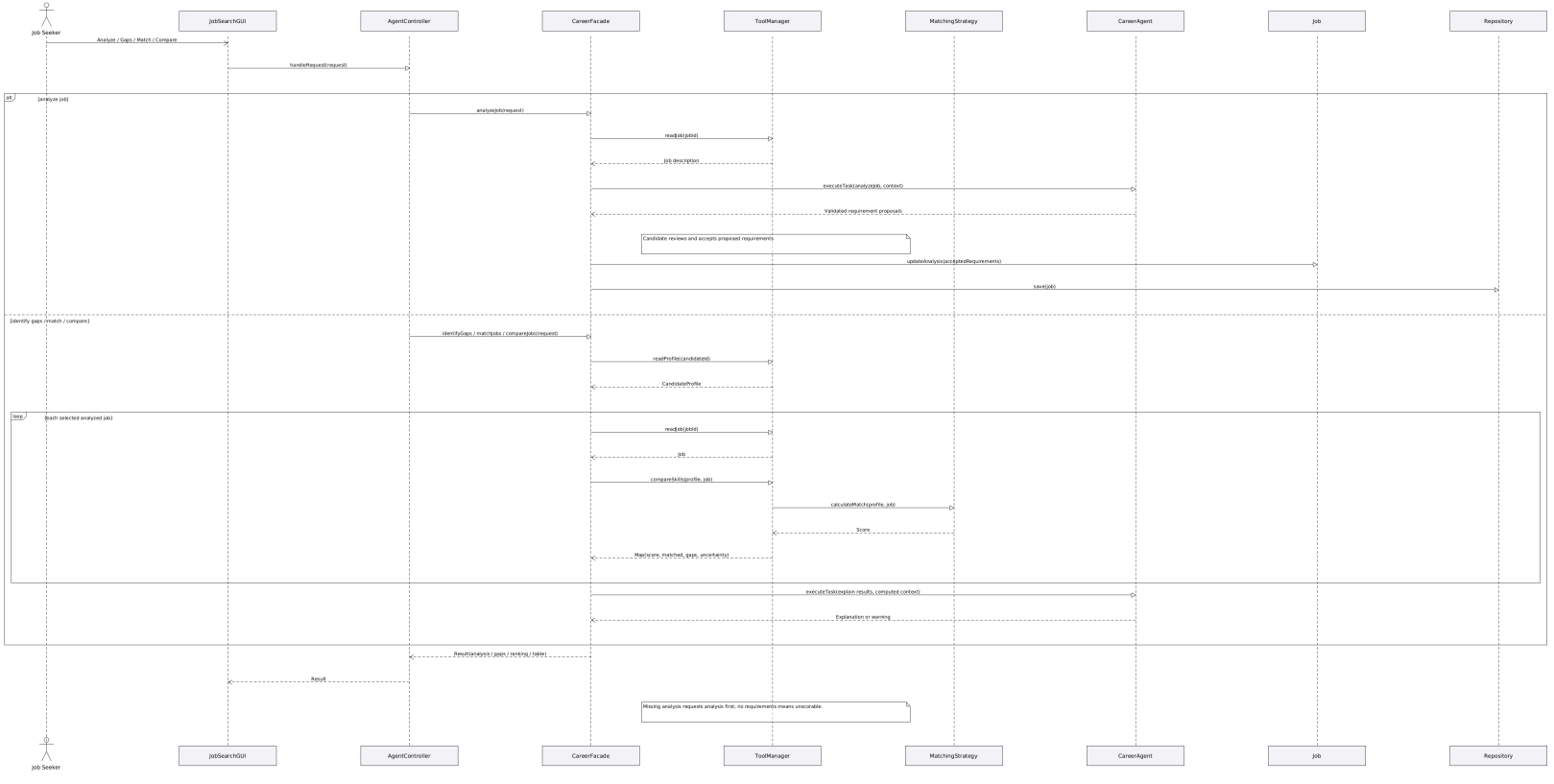
F02-F03. ResumeParser is DocumentParserAdapter. Skills are confirmed before profile changes. SD07 expands AI execution.



Solid arrows: calls / user actions. Dashed arrows: returns. CLI uses the same controller and workflows.

SD02 - Job Analysis, Gaps, Matching and Comparison

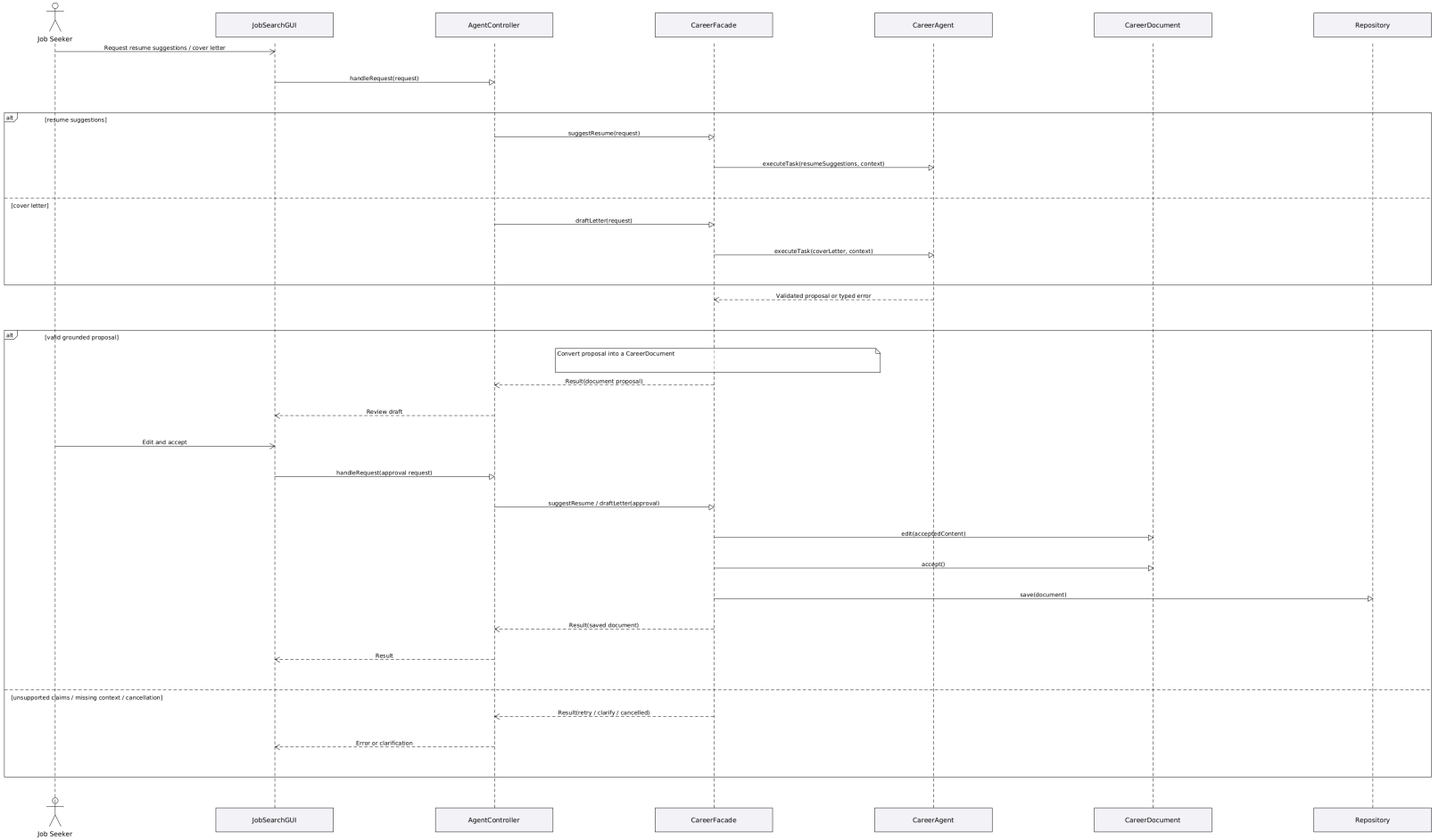
F05-F08. Analysis is reviewed before saving; AI explanation failure retains computed scores and tables.



Solid arrows: calls / user actions. Dashed arrows: returns. CLI uses the same controller and workflows.

SD03 - Resume Suggestions and Cover-Letter Drafting

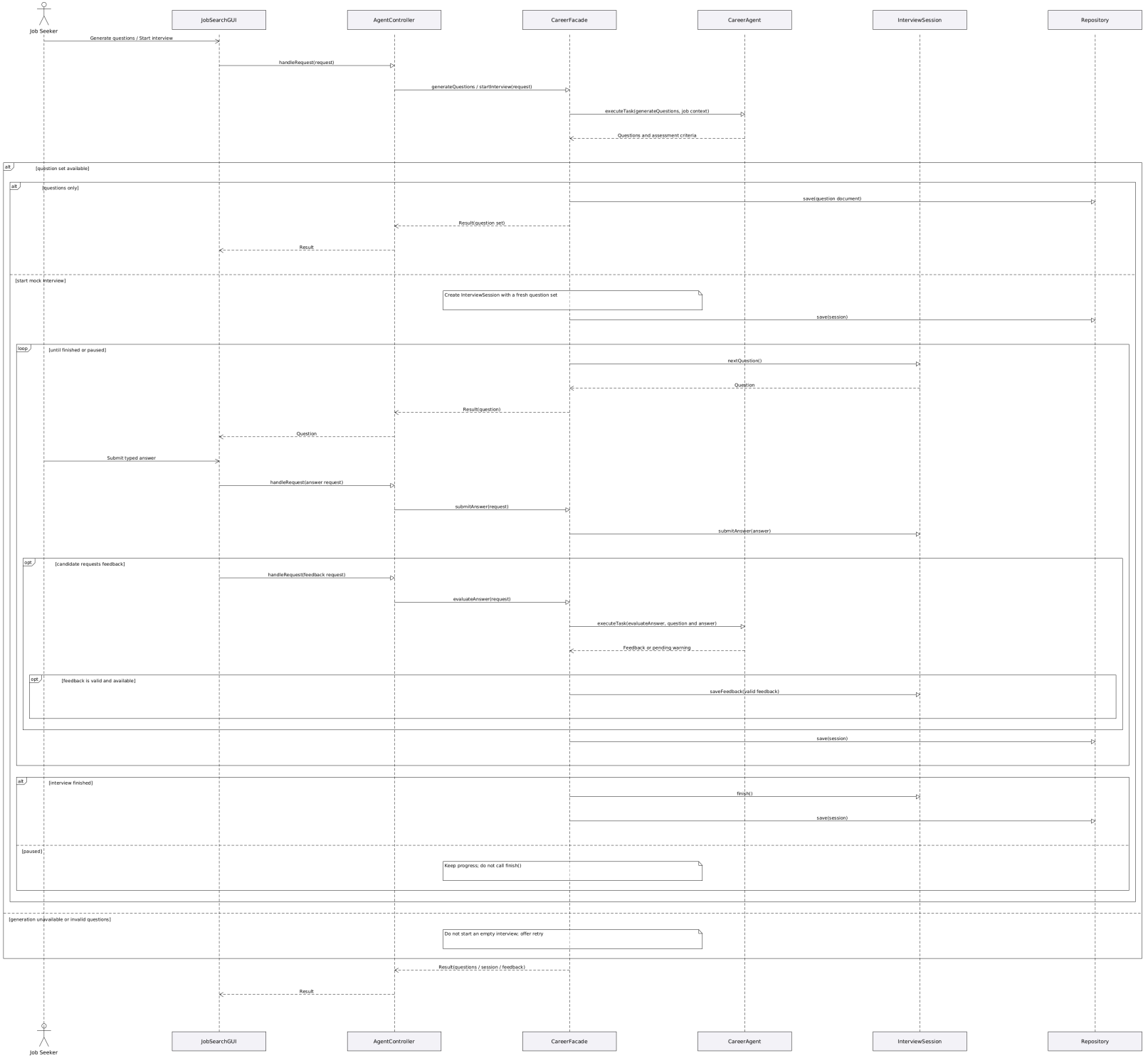
F09-F10. The candidate reviews and accepts the document. SD07 expands the reusable agent loop.



Solid arrows: calls / user actions. Dashed arrows: returns. CLI uses the same controller and workflows.

SD04 - Questions, Mock Interview and Feedback

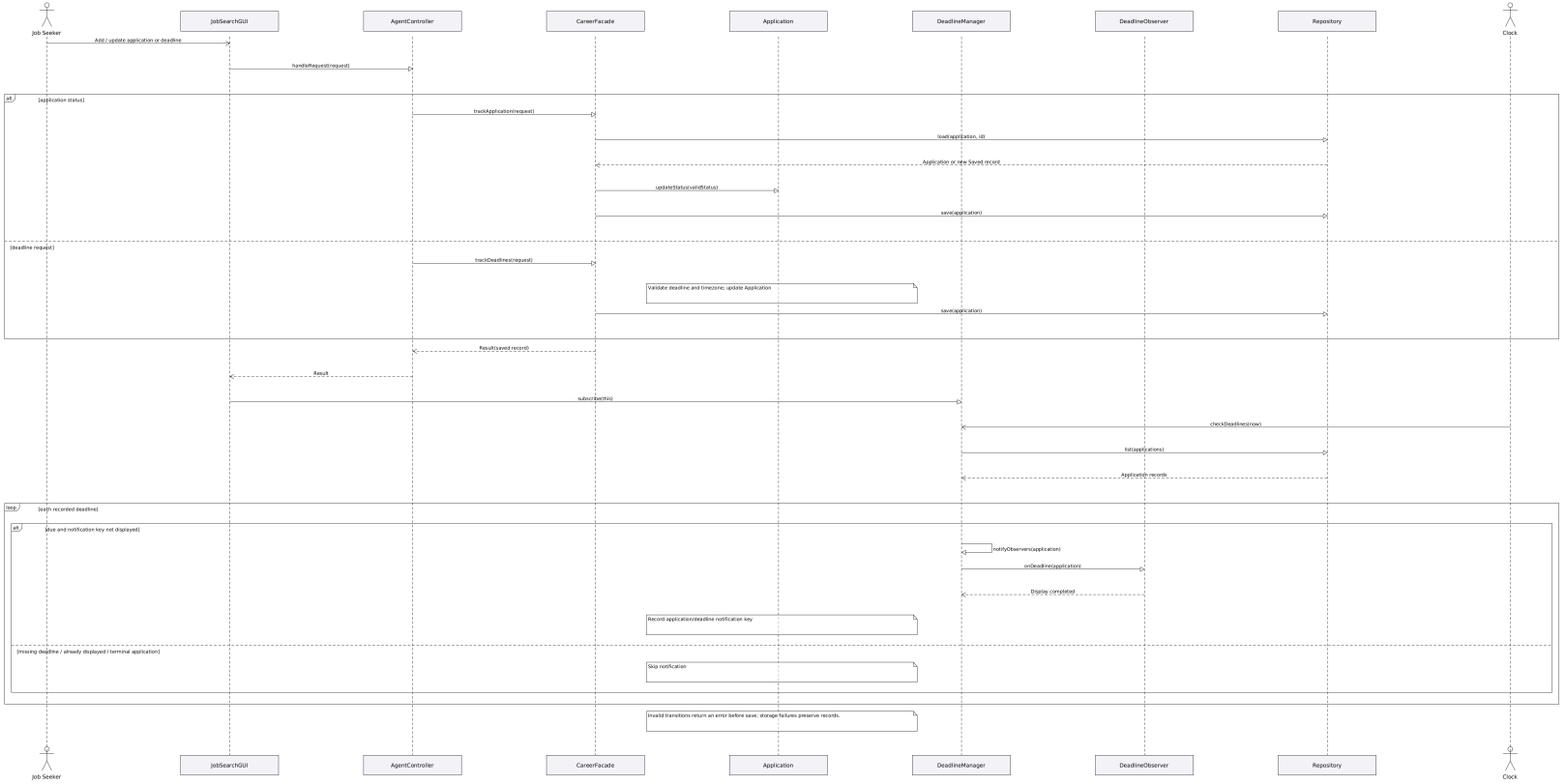
F11-F13. Empty answers require retry or explicit skip. Feedback failure retains the stored answer.



Solid arrows: calls / user actions. Dashed arrows: returns. CLI uses the same controller and workflows.

SD05 - Application Tracking and Deadlines

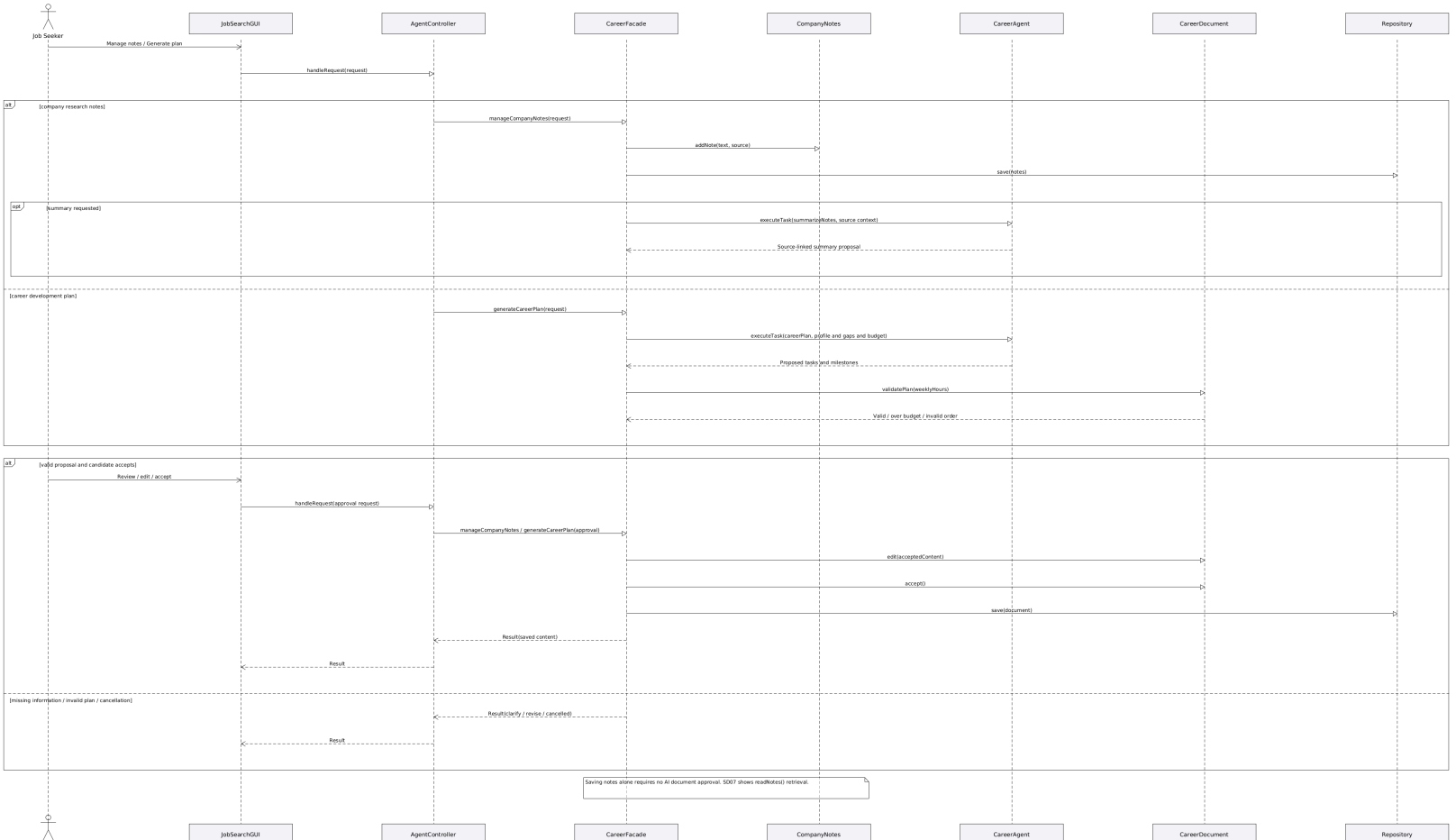
F14-F15. DeadlineObserver is the active GUI or CLI. Clock represents the scheduled check.



Solid arrows: calls / user actions. Dashed arrows: returns. CLI uses the same controller and workflows.

SD06 - Company Notes and Career Development Plan

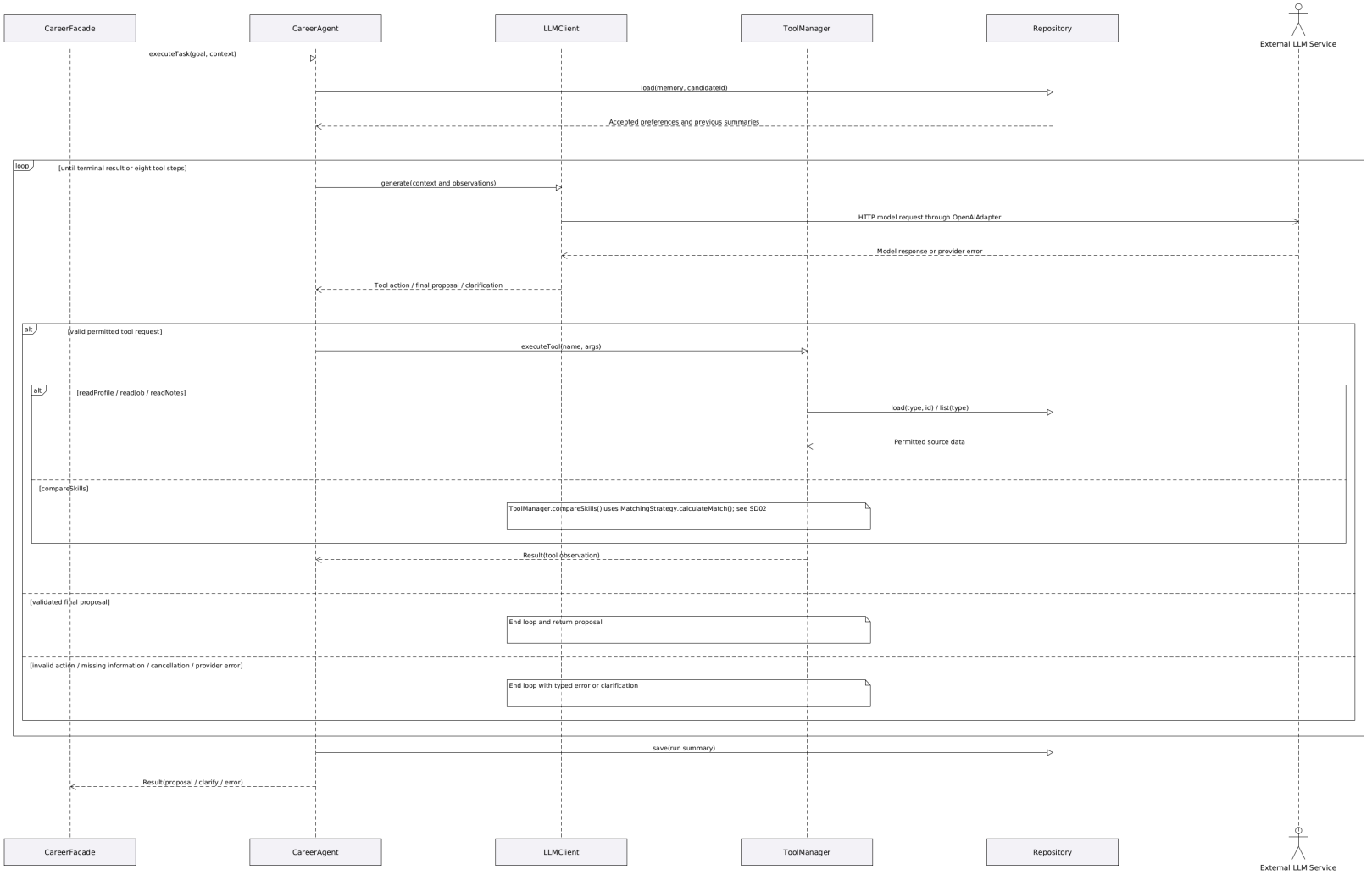
F16-F17. Notes summarize supplied sources. Invalid or over-budget plans cannot be accepted.



Solid arrows: calls / user actions. Dashed arrows: returns. CLI uses the same controller and workflows.

SD07 - Agent Planning, Tools, AI Service and Memory

Shared AI behavior. ToolManager permits retrieval and skill comparison; maxSteps = 8.



Solid arrows: calls / user actions. Dashed arrows: returns. CLI uses the same controller and workflows.

Task 3: Feature-to-Design Traceability

Feature	Description	Type	Related UseCase	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01	Manage profile	Deterministic	UC01	CandidateProfile, Repository	CareerFacade.updateProfile(), CandidateProfile.updateProfile(), save()	SD00	MVC, Facade
F02	Import Resume	Deterministic	UC02	ResumeParser, DocumentParserAdapter, Resume	importResume(), extractText(), hasText(), save()	SD01	MVC, Facade, Adapter
F03	Extract skills	AI-based	UC03	CareerAgent, LLMClient, CandidateProfile	extractSkills(), executeTask(), generate(), updateSkills()	SD01, SD07	MVC, Facade, Adapter
F04	Import Job	Deterministic	UC04	Job, Repository	importJob(), save()	SD00	MVC, Facade
F05	Analyze requirements	AI-based	UC05	CareerAgent, ToolManager, Job, LLMClient	analyzeJob(), readJob(), executeTask(), updateAnalysis	SD02, SD07	MVC, Facade, Strategy, Adapter
F06	Identify gaps	Hybrid	UC06	ToolManager, MatchingStrategy, CareerAgent	identifyGaps(), compareSkills(), executeTask()	SD02, SD07	MVC, Facade, Strategy, Adapter
F07	Rank jobs	Hybrid	UC07	ToolManager, MatchingStrategy, CareerAgent	matchJobs(), compareSkills(), calculateMatch()	SD02, SD07	MVC, Facade, Strategy, Adapter
F08	Compare jobs	Hybrid	UC08	ToolManager, CareerAgent, Job	compareJobs(), readJob(), compareSkills(), executeTask()	SD02, SD07	MVC, Facade, Strategy, Adapter
F09	Suggest resume changes	AI-based	UC09	CareerAgent, CareerDocument, LLMClient	suggestResume(), executeTask(), edit(), accept()	SD03, SD07	MVC, Facade, Adapter
F10	Draft cover letter	AI-based	UC10	CareerAgent, CareerDocument, LLMClient	draftLetter(), executeTask(), edit(), accept()	SD03, SD07	MVC, Facade, Adapter
F11	Generate questions	AI-based	UC11	CareerAgent, CareerDocument, LLMClient	generateQuestions(), executeTask(), generate()	SD04, SD07	MVC, Facade, Adapter
F12	Conduct interview	Hybrid	UC12	InterviewSession, CareerAgent	startInterview(), nextQuestion(), submitAnswer(), finish()	SD04, SD07	MVC, Facade, Adapter

Feature	Description	Type	Related UseCase	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F13	Evaluate answers	AI-based	UC13	CareerAgent, InterviewSession, LLMClient	evaluateAnswer(), executeTask(), saveFeedback()	SD04, SD07	MVC, Facade, Adapter
F14	Track application	Deterministic	UC14	Application, Repository	trackApplication(), updateStatus(), save()	SD05	MVC, Facade
F15	Track deadlines	Deterministic	UC15	DeadlineManager, DeadlineObserver, Application	trackDeadlines(), checkDeadlines(), notifyObservers(), onDeadline()	SD05	MVC, Facade, Adapter, Observer
F16	Manage company notes	Hybrid	UC16	CompanyNotes, CareerAgent, ToolManager	manageCompanyNotes(), addNote(), readNotes(), executeTask()	SD06, SD07	MVC Facade, Adapter
F17	Generate career plan	AI-based	UC17	CareerAgent, CareerDocument, ToolManager	generateCareerPlan(), executeTask(), validatePlan(), accept()	SD06, SD07	MVC, Facade, Adapter

Task 4: Explain How Each Feature is Realized

F01 – Candidate Profile Management

Related Use Case: UC01 - Manage Candidate Profile
Related Sequence Diagram: SD00 - Profile Management and Job Import

Classes involved:

- CareerFacade - coordinates the feature workflow and returns its result.
- CandidateProfile - stores candidate information and confirmed skills.
- Repository - loads and saves local JSON records.

Important methods:

- CareerFacade.updateProfile()
- CandidateProfile.updateProfile()
- Repository.save()

Execution:
When the user selects Save Profile in the GUI, submitRequest() sends the user's profile information to AgentController. The controller invokes CareerFacade.updateProfile(). This verifies the user's information and calls the method CandidateProfile.updateProfile() and updates the candidate's profile. The method Repository.save() stores the updated CandidateProfile object and returns the result to the GUI.

F02 – Candidate Profile Management

Related Use Case: UC02 - Resume Import
Related Sequence Diagram: SD01 - Resume Import and Skill Extraction

Classes involved:

- Resume - the imported resume and its extracted text
- ResumeParser - defines the resume text-extraction contract
- CareerFacade - coordinates the feature’s workflow and returns its result.
- Repository - loads and saves local JSON records.
- DocumentParserAdapter - adapts the PDF/DOCX libraries to ResumeParser

Important methods:

- CareerFacade.importResume()
- ResumeParser.extractText()
- Resume.hasText()
- Repository.save()

Execution:
When the user selects Import Resume in the GUI, submitRequest() sends the selected file to AgentController. The controller invokes CareerFacade.importResume(). The facade uses ResumeParser.extractText(), implemented by DocumentParserAdapter, to extract the resume text. The result is converted into a Resume object and checked using Resume.hasText(). Repository.save() stores the resume, and the import result is returned to the GUI.

F03 – Resume Skill Extraction

Related Use Case: UC03 - Extract Resume Skills
Related Sequence Diagram: SD01 - Resume Import and Skill Extraction
SD07 - Agent Planning, Tools, and Memory

Classes involved:

- CareerAgent - plans the task, retrieves context and coordinates model use
- ToolManager - provides permitted retrieval and skill-comparison tools.
- LLMClient - defines communication with the selected LLM.
- CandidateProfile - stores candidate information and confirmed skills.

Important methods:

- CareerFacade.extractSkills()
- CareerAgent.executeTask()
- LLMClient.generate()
- CandidateProfile.updateSkills()

Execution:
When the user selects Extract Skills in the GUI, submitRequest() sends the selected resume information to AgentController. The controller invokes CareerFacade.extractSkills(). The facade calls CareerAgent.executeTask(), which uses ToolManager to gather the required context. Relevant resume text is passed to LLMClient.generate(), which proposes skills supported by the resume. The proposed skills are returned to the GUI for confirmation, after which CandidateProfile.updateSkills() updates the candidate's skills.

F04 – Job Import

Related Use Case: UC04 - Import Job
Related Sequence Diagram: SD00 - Profile Management and Job Import

Classes involved:

- CareerFacade - coordinates the feature's workflow and returns its result.
- Job - provides permitted retrieval and skill-comparison tools.
- Repository- loads and saves local JSON records.

Important methods:

- CareerFacade.importJob()
- Repository.save()

Execution:

When the user selects Import Job in the GUI, submitRequest() sends the job details and description to AgentController. The controller invokes CareerFacade.importJob(). The facade validates the information and checks whether the job already exists.The information is converted into a new or updated Job object and stored through Repository.save(). The saved job is returned to the GUI and displayed as a job card.

F05 – Job Description Analysis

Related Use Case: UC05 - Extract Resume Skills
Related Sequence Diagram: SD02 - Job Analysis, Gaps, Matching, and Comparison
SD07 - Agent Planning, Tools, and Memory

Classes involved:

- CareerAgent - plans the task, retrieves context and coordinates model use
- ToolManager - provides permitted retrieval and skill-comparison tools
- LLMClient - defines communication with the selected LLM
- Job - stores the job description and reviewed requirements

Important methods:

- CareerFacade.analyzeJob()
- CareerAgent.executeTask()
- Job.updateAnalysis()
- ToolManager.readJob()

Execution:

When the user selects Analyze Job in the GUI, submitRequest() sends the selected job identifier to AgentController. The controller invokes CareerFacade.analyzeJob(). The facade uses ToolManager.readJob() to retrieve the job and calls CareerAgent.executeTask(). Relevant job information is passed to LLMClient, which proposes the job's skill requirements. After review and acceptance, Job.updateAnalysis() records the requirements, and the analysis is returned to the GUI.

F06 – Skill-Gap Identification

Related Use Case: UC06 - Identify Skill Gaps
Related Sequence Diagram: SD02 - Job Analysis, Gaps, Matching, and Comparison
SD07 - Agent Planning, Tools, and Memory

Classes involved:

- CareerAgent - plans the task, retrieves context and coordinates model use
- ToolManager - provides permitted retrieval and skill-comparison tools
- MatchingStrategy - defines the interchangeable match-score calculation

Important methods:

- CareerFacade.identifyGaps()
- CareerAgent.executeTask()
- ToolManager.compareSkills()

Execution:
When the user selects Identify Skill Gaps in the GUI, submitRequest() sends the selected candidate and job information to AgentController. The controller invokes CareerFacade.identifyGaps(). The facade uses ToolManager.compareSkills() to identify matching, missing, and uncertain skills. These results are passed to CareerAgent.executeTask(), which obtains an explanation from LLMClient. The skill gaps and their explanation are returned to the GUI.

F07 – Job Matching

Related Use Case: UC07 - Match Jobs
Related Sequence Diagram: SD02 - Job Analysis, Gaps, Matching, and Comparison
SD07 - Agent Planning, Tools, and Memory

Classes involved:

- CareerAgent - plans the task, retrieves context and coordinates model use
- ToolManager - provides permitted retrieval and skill-comparison tools
- MatchingStrategy - defines the interchangeable match-score calculation
- WeightedMatching - calculates weighted coverage of required and preferred skills
- RequiredSkillMatching - applied the required-skill policy before weighted calculation

Important methods:

- CareerFacade.matchJobs()
- MatchingStrategy.calculateMatch()
- ToolManager.compareSkills()

Execution:
When the user selects Match Jobs in the GUI, submitRequest() sends the selected jobs and matching preferences to AgentController. The controller invokes CareerFacade.matchJobs(). The facade calls the method ToolManager.compareSkills() to compare the candidate's skills with each job's requirements. MatchingStrategy.calculateMatch() calculates the match scores. The jobs are then ranked according to these scores, and the matching results are returned to the GUI.

F08 – Job Matching

Related Use Case: UC08 - Compare Jobs
Related Sequence Diagram: SD02 - Job Analysis, Gaps, Matching, and Comparison
SD07 - Agent Planning, Tools, and Memory

Classes involved:

- CareerAgent - plans the task, retrieves context and coordinates model use
- ToolManager - provides permitted retrieval and skill-comparison tools
- CareerFacade - coordinates the feature’s workflow and returns its result.
- RequiredSkillMatching - applied the required-skill policy before weighted calculation

Important methods:

- CareerFacade.compareJobs()
- ToolManager.readJob()
- CareerAgent.executeTask()
- ToolManager.compareSkills()

Execution:
When the user selects Compare Jobs in the GUI, submitRequest() sends the selected job identifiers to AgentController. The controller invokes CareerFacade.compareJobs(). The facade uses ToolManager.readJob() to retrieve the jobs and ToolManager.compareSkills() to compare the job’s requirements with the candidate's skills. If an AI summary is requested, CareerAgent.executeTask() passes information to LLMClient. As a result, the comparison table and requested summary are returned to the GUI.

F09 – Customized Resume Suggestions

Related Use Case: UC09 - Generate Resume Suggestions
Related Sequence Diagram: SD03 - Resume Suggestions and Cover-Letter Drafting
SD07 - Agent Planning, Tools, and Memory

Classes involved:

- CareerAgent - plans the task, retrieves context and coordinates model use
- CareerDocument - stores generated content, edits, and acceptances
- LLMClient - defines communication with the selected LLM

Important methods:

- CareerFacade.suggestResume()
- CareerAgent.executeTask()
- CareerDocument.accept()
- CareerDocument.edit()

Execution:
When the user selects Generate Resume Suggestions in the GUI, submitRequest() sends the selected resume and job information to AgentController. The controller invokes CareerFacade.suggestResume(). The facade calls CareerAgent.executeTask(), which gathers the required context and passes it to LLMClient. The generated suggestions are converted into a CareerDocument object and returned to the GUI. The user edits and accepts the suggestions through CareerDocument.edit() and CareerDocument.accept().

F10 – Cover-Letter Drafting

Related Use Case: UC10 - Draft Cover Lettter
Related Sequence Diagram: SD03 - Resume Suggestions and Cover-Letter Drafting
SD07 - Agent Planning, Tools, and Memory

Classes involved:

- CareerAgent - plans the task, retrieves context and coordinates model use
- CareerDocument - stores generated content, edits, and acceptances
- LLMClient - defines communication with the selected LLM

Important methods:

- CareerFacade.draftLetter()
- CareerAgent.executeTask()
- CareerDocument.accept()
- CareerDocument.edit()

Execution:
When the user selects Draft Cover Letter in the GUI, submitRequest() sends the candidate information, selected job, and writing preferences to AgentController. The controller invokes CareerFacade.draftLetter(). The facade calls CareerAgent.executeTask(), which passes the relevant context to LLMClient to generate a proposed cover letter. The result is converted into a

CareerDocument object and returned to the GUI. The user revises and approves the draft through CareerDocument.edit() and CareerDocument.accept().

F11 – Interview Question Generation

Related Use Case: UC11 - Generate Interview Questions
Related Sequence Diagram: SD04 - Questions, Mock Interview, and Feedback
SD07 - Agent Planning, Tools, and Memory

Classes involved:

- CareerAgent - plans the task, retrieves context, and coordinates model use
- CareerDocument - stores generated content, edits, and acceptances
- LLMClient - defines communication with the selected LLM

Important methods:

- CareerFacade.generateQuestions()
- CareerAgent.executeTask()
- LLMClient.generate()

Execution:
When the user selects Generate Interview Questions in the GUI, submitRequest() sends the selected job and interview preferences to AgentController. The controller invokes CareerFacade.generateQuestions(). The facade calls CareerAgent.executeTask() to gather the required candidate and job information. Relevant context is passed to LLMClient.generate(), which generates interview questions and assessment criteria. The validated result is converted into a CareerDocument object and returned to the GUI.

F12 – Interview Question Generation

Related Use Case: UC12 - Conduct Mock Interview
Related Sequence Diagram: SD04 - Questions, Mock Interview, and Feedback
SD07 - Agent Planning, Tools, and Memory

Classes involved:

- CareerAgent - plans the task, retrieves context, and coordinates model use
- CareerFacade - coordinates the feature’s workflow and returns its result.
- InterviewSession - stores interview questions, answers, feedback, and progress

Important methods:

- CareerFacade.startInterview()

- InterviewSession.nextQuestion()
- InterviewSession.submitAnswer()
- InterviewSession.finish()

Execution:
When the user selects Start Mock Interview in the GUI, submitRequest() sends the settings of the selected job and interview to AgentController. The controller invokes CareerFacade.startInterview(). The facade uses CareerAgent to obtain interview questions and creates an InterviewSession object. The InterviewSession.nextQuestion() method provides the user with every question and InterviewSession.submitAnswer() records the user's answer for submission. When the interview is over, the InterviewSession.finish() methods finishes the session, and the final results are returned to the GUI.

F13 – Interview Answer Feedback

Related Use Case: UC13 - Evaluate Interview Answers
Related Sequence Diagram: SD04 - Questions, Mock Interview, and Feedback
SD07 - Agent Planning, Tools, and Memory

Classes involved:

- CareerAgent - plans the task, retrieves context, and coordinates model use
- InterviewSession - stores interview questions, answers, feedback, and progress
- LLMClient - defines communication with the selected LLM

Important methods:

- CareerFacade.evaluateAnswer()
- CareerAgent.executeTask()
- InterviewSession.saveFeedback()

Execution:
When the user selects Get Answer Feedback in the GUI, submitRequest() sends the selected interview session and answer information to AgentController. The controller invokes CareerFacade.evaluateAnswer() to check the candidate’s answers according to the evaluation criteria. The facade calls CareerAgent.executeTask(), which gathers the interview question, stored answer, and evaluation criteria. Relevant context is passed to LLMClient, which generates feedback and improvement suggestions. InterviewSession.saveFeedback() records the validated feedback, and the result is returned to the GUI.

F14 – Application Tracking

Related Use Case: UC14 - Track Applications
Related Sequence Diagram: SD05 - Applications and Deadlines

Classes involved:

- CareerFacade - coordinates the feature's workflow and returns its result.
- Repository- loads and saves local JSON records.
- Application - stores application status, dates, notes, and history

Important methods:

- CareerFacade.trackApplication()
- Application.updateStatus()
- Repository.save()

Execution:
When the user selects Save Application or Update Status in the GUI, submitRequest() sends the application information to AgentController. The controller invokes CareerFacade.trackApplication(). The facade creates or retrieves an Application object and uses Application.updateStatus() when a status change is requested. The application checks the permitted transition and records its status history. Repository.save() stores the application, and the updated tracking information is returned to the GUI.

F15 – Deadline Tracking

Related Use Case: UC15 - Track Deadlines
Related Sequence Diagram: SD05 - Applications and Deadlines

Classes involved:

- DeadlineManager - checks deadlines and notifies subscribed observers
- DeadlineObserver - defines the deadline notification callback
- Application - stores application status, dates, notes, and history
- JobSearchGUI - displays deadline notifications in the graphical interface
- JobSearchCLI - displays deadline notifications in the command-line interface

Important methods:

- CareerFacade.trackDeadlines()
- DeadlineManager.checkDeadlines()
- DeadlineManager.notifyObservers()

- DeadlineObserver.onDeadline()

Execution:
When the user selects Check Deadlines in the GUI, submitRequest() sends the request to AgentController. The controller invokes CareerFacade.trackDeadlines(). The facade calls DeadlineManager.checkDeadlines() to compare the stored deadlines with the current time in order to detect any past or upcoming deadlines. DeadlineManager.notifyObservers() sends all of the important and relevant reminders to the subscribed observers through DeadlineObserver.onDeadline(). The deadline information and reminders are displayed in the GUI.

F16 – Company Research Notes

Related Use Case: UC16 - Manage Company Notes
Related Sequence Diagram: SD06 - Company Notes and Career Plan
SD07 - Agent Planning, Tools, and Memory

Classes involved:
- CompanyNotes - checks deadlines and notifies subscribed observers
- CareerAgent - plans the task, retrieves context, and coordinates model use
- CareerDocument - stores generated content, edits, and acceptances
- ToolManager - provides permitted retrieval and skill-comparison tools

Important methods:
- CareerFacade.manageCompanyNotes()
- CompanyNotes.addNote()
- ToolManager.readNotes()
- CareerAgent.executeTask()

Execution:
When the user selects Save Company Note in the GUI, submitRequest() sends the company name, note, and source information to AgentController. The controller invokes CareerFacade.manageCompanyNotes(). The facade calls CompanyNotes.addNote() to record the supplied information. If a summary is requested, ToolManager.readNotes() retrieves the notes, and CareerAgent.executeTask() passes the context to LLMClient. The generated summary is converted into a CareerDocument object and returned to the GUI for review.

F17 – Career Development Plan

Related Use Case: UC17 - Generate Career Plan
Related Sequence Diagram: SD06 - Company Notes and Career Plan
SD07 - Agent Planning, Tools, and Memory

Classes involved:

- CareerAgent - plans the task, retrieves context, and coordinates model use
- CareerDocument - stores generated content, edits, and acceptances
- ToolManager - provides permitted retrieval and skill-comparison tools

Important methods:

- CareerFacade.generateCareerPlan()
- CareerAgent.executeTask()
- CareerDocument.validatePlan()
- CareerDocument.accept()

Execution:
When the user selects Save Company Note in the GUI, submitRequest() sends the company name, note, and source information to AgentController. The controller invokes CareerFacade.manageCompanyNotes(). The facade calls CompanyNotes.addNote() to record the supplied information. If a summary is requested, ToolManager.readNotes() retrieves the notes, and CareerAgent.executeTask() passes the context to LLMClient. The generated summary is converted into a CareerDocument object and returned to the GUI for review.